

Python - Change Dictionary Items

Change Dictionary Items

Changing dictionary items in Python refers to modifying the values associated with specific keys within a dictionary. This can involve updating the value of an existing key, adding a new key-value pair, or removing a key-value pair from the dictionary.

Dictionaries are mutable, meaning their contents can be modified after they are created.

Modifying Dictionary Values

Modifying values in a Python dictionary refers to changing the value associated with an existing key. To achieve this, you can directly assign a new value to that key.

Example

In the following example, we are defining a dictionary named "person" with keys 'name', 'age', and 'city' and their corresponding values. Then, we modify the value associated with the key 'age' to 26 –

```
# Initial dictionary
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}
# Modifying the value associated with the key 'age'
person['age'] = 26
print(person)
```

It will produce the following output –

```
{'name': 'Alice', 'age': 26, 'city': 'New York'}
```

Updating Multiple Dictionary Values

If you need to update multiple values in a dictionary at once, you can use the `update()` method. This method is used to update a dictionary with elements from another dictionary or an iterable of key-value pairs.

The `update()` method adds the key-value pairs from the provided dictionary or iterable to the original dictionary, overwriting any existing keys with the new values if they already exist in the original dictionary.

Example

In the example below, we are using the `update()` method to modify the values associated with the keys `'age'` and `'city'` in the `'persons'` dictionary –

```
# Initial dictionary
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}
# Updating multiple values
person.update({'age': 26, 'city': 'Los Angeles'})
print(person)
```

We get the output as shown below –

```
{'name': 'Alice', 'age': 26, 'city': 'Los Angeles'}
```

Conditional Dictionary Modification

Conditional modification in a Python dictionary refers to changing the value associated with a key only if a certain condition is met.

You can use an `if` statement to check whether a certain condition is true before modifying the value associated with a key.

Example

In this example, we conditionally modify the value associated with the key `'age'` to `'26'` if the current value is `'25'` in the `'persons'` dictionary –

```
# Initial dictionary
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}
# Conditionally modifying the value associated with 'age'
if person['age'] == 25:
    person['age'] = 26
print(person)
```

The output obtained is as shown below –

```
{'name': 'Alice', 'age': 26, 'city': 'New York'}
```

Modify Dictionary by Adding New Key-Value Pairs

Adding new key-value pairs to a Python dictionary refers to inserting a new key along with its corresponding value into the dictionary.

This process allows you to dynamically expand the data stored in the dictionary by including additional information as needed.

Example: Using Assignment Operator

You can add a new key-value pair to a dictionary by directly assigning a value to a new key as shown below. In the example below, the key 'city' with the value 'New York' is added to the 'person' dictionary –

```
# Initial dictionary
person = {'name': 'Alice', 'age': 25}
# Adding a new key-value pair 'city': 'New York'
person['city'] = 'New York'
print(person)
```

The result produced is as follows –

```
{'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Example: Using the.setdefault() Method

You can use the `setdefault()` method to add a new key-value pair to a dictionary if the key does not already exist.

In this example, the `setdefault()` method adds the new key 'city' with the value 'New York' to the 'person' dictionary only if the key 'city' does not already exist –

```
# Initial dictionary
person = {'name': 'Alice', 'age': 25}
# Adding a new key-value pair 'city': 'New York'
person.setdefault('city', 'New York')
print(person)
```

Following is the output of the above code –

```
{'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Modify Dictionary by Removing Key-Value Pairs

Removing key-value pairs from a Python dictionary refers to deleting specific keys along with their corresponding values from the dictionary.

This process allows you to selectively remove data from the dictionary based on the keys you want to eliminate.

Example: Using the del Statement

You can use the **del** statement to remove a specific key-value pair from a dictionary. In this example, the `del` statement removes the key 'age' along with its associated value from the 'person' dictionary –

```
# Initial dictionary
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}
# Removing the key-value pair associated with the key 'age'
```

```
del person['age']  
print(person)
```

Output of the above code is as shown below –

```
{'name': 'Alice', 'city': 'New York'}
```

Example: Using the pop() Method

You can also use the **pop()** method to remove a specific key-value pair from a dictionary and return the value associated with the removed key.

In here, the pop() method removes the key 'age' along with its associated value from the 'person' dictionary –



```
# Initial dictionary  
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}  
# Removing the key-value pair associated with the key 'age'  
removed_age = person.pop('age')  
  
print(person)  
print("Removed age:", removed_age)
```

It will produce the following output –

```
{'name': 'Alice', 'city': 'New York'}  
Removed age: 25
```

Example: Using the popitem() Method

You can use the **popitem()** method as well to remove the last key-value pair from a dictionary and return it as a tuple.

Now, the popitem() method removes the last key-value pair from the 'person' dictionary and returns it as a tuple –



```
# Initial dictionary
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}
# Removing the last key-value pair
removed_item = person.popitem()

print(person)
print("Removed item:", removed_item)
```

We get the output as shown below –

```
{'name': 'Alice', 'age': 25}
Removed item: ('city', 'New York')
```

TOP TUTORIALS

- Python Tutorial
- Java Tutorial
- C++ Tutorial
- C Programming Tutorial
- C# Tutorial
- PHP Tutorial
- R Tutorial
- HTML Tutorial
- CSS Tutorial
- JavaScript Tutorial
- SQL Tutorial

TRENDING TECHNOLOGIES

- Cloud Computing Tutorial
- Amazon Web Services Tutorial
- Microsoft Azure Tutorial
- Git Tutorial
- Ethical Hacking Tutorial
- Docker Tutorial
- Kubernetes Tutorial

DSA Tutorial

Spring Boot Tutorial

SDLC Tutorial

Unix Tutorial

CERTIFICATIONS

Business Analytics Certification

Java & Spring Boot Advanced Certification

Data Science Advanced Certification

Cloud Computing And DevOps

Advanced Certification In Business Analytics

Artificial Intelligence And Machine Learning

DevOps Certification

Game Development Certification

Front-End Developer Certification

AWS Certification Training

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online Go Compiler

Online C Compiler

Online C++ Compiler

Online C# Compiler



Chapters ▾

Categories ≡

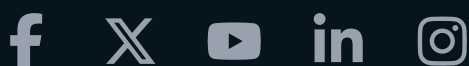
Online Bash Terminal

Online SQL Compiler

Online Html Editor

ABOUT US | OUR TEAM | CAREERS | JOBS | CONTACT US | TERMS OF USE |

PRIVACY POLICY | REFUND POLICY | COOKIES POLICY | FAQ'S



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2025. All Rights Reserved.